

F1 Lap Time Prediction - Exploratory Data Analysis (<https://docs.fastf1.dev/>)

This project aims to predict a Formula 1 driver's next lap time using information available at the end of the current lap. Using race data collected through FastF1, I will explore how previous lap times, tyre conditions and weather could help a machine learning regression model predict race pace.

In this notebook, I check season coverage and explore Monza 2024 race data. I use plots and summary statistics to identify patterns, missing values and unusual laps, helping me decide which features and filtering steps to use before modelling.

Imports

```
In [20]: import pandas as pd
import matplotlib.pyplot as plt
import fastf1
from pathlib import Path
```

Loading Events Calendar

```
In [21]: project_root=Path.cwd()
if project_root.name=="notebooks":
    project_root=project_root.parent
events=pd.read_csv(
    project_root / "data" / "events_2022_2025.csv",
    parse_dates=["EventDate"]
)

events[["Season", "RoundNumber", "EventDate", "EventName"]]
```

Out[21]:

	Season	RoundNumber	EventDate	EventName
0	2022	1	2022-03-20	Bahrain Grand Prix
1	2022	2	2022-03-27	Saudi Arabian Grand Prix
2	2022	3	2022-04-10	Australian Grand Prix
3	2022	4	2022-04-24	Emilia Romagna Grand Prix
4	2022	5	2022-05-08	Miami Grand Prix
...	...	...	...	...
87	2025	20	2025-10-26	Mexico City Grand Prix
88	2025	21	2025-11-09	São Paulo Grand Prix
89	2025	22	2025-11-22	Las Vegas Grand Prix
90	2025	23	2025-11-30	Qatar Grand Prix
91	2025	24	2025-12-07	Abu Dhabi Grand Prix

92 rows × 4 columns

Loading all race laps

```
In [22]: laps = pd.read_pickle(
    project_root / "data" / "race_laps_2022_2025.pkl"
)
print(f"Loaded {len(laps):,} saved laps.")
```

Loaded 101,290 saved laps.

Event Coverage Across Seasons

```
In [23]: event_presence=pd.crosstab(
    events["EventName"],
    events["Season"]
)

event_presence=event_presence.reindex(
    columns=range(2022,2026),
    fill_value=0
).gt(0)

coverage=event_presence.copy()

coverage["MissingYears"]=event_presence.apply(
    lambda row: ",".join(
        str(year) for year in row.index if not row[year]
    ),
    axis=1
```

)

coverage

Out[23]:

Season	2022	2023	2024	2025	MissingYears
EventName					
Abu Dhabi Grand Prix	True	True	True	True	
Australian Grand Prix	True	True	True	True	
Austrian Grand Prix	True	True	True	True	
Azerbaijan Grand Prix	True	True	True	True	
Bahrain Grand Prix	True	True	True	True	
Belgian Grand Prix	True	True	True	True	
British Grand Prix	True	True	True	True	
Canadian Grand Prix	True	True	True	True	
Chinese Grand Prix	False	False	True	True	2022,2023
Dutch Grand Prix	True	True	True	True	
Emilia Romagna Grand Prix	True	False	True	True	2023
French Grand Prix	True	False	False	False	2023,2024,2025
Hungarian Grand Prix	True	True	True	True	
Italian Grand Prix	True	True	True	True	
Japanese Grand Prix	True	True	True	True	
Las Vegas Grand Prix	False	True	True	True	2022
Mexico City Grand Prix	True	True	True	True	
Miami Grand Prix	True	True	True	True	
Monaco Grand Prix	True	True	True	True	
Qatar Grand Prix	False	True	True	True	2022
Saudi Arabian Grand Prix	True	True	True	True	
Singapore Grand Prix	True	True	True	True	
Spanish Grand Prix	True	True	True	True	
São Paulo Grand Prix	True	True	True	True	
United States Grand Prix	True	True	True	True	

Findings:

Of the 25 distinct Grand Prix events, 20 appear in all four seasons. The French Grand Prix appears only in 2022, limiting comparisons over time. All events remain in the general EDA, while comparisons across all four seasons can focus on recurring events.

Download completeness and data quality

```
In [24]: expected_events=events[
        ["Season", "EventName", "RoundNumber"]
        ].drop_duplicates()

        loaded_events=laps[
        ["RoundNumber", "Season"]
        ].drop_duplicates()
```

```
In [25]: download_check=expected_events.merge(
        loaded_events,
        on=["Season", "RoundNumber"],
        how="left",
        indicator=True
        )

        missing_events=download_check.loc[
        download_check["_merge"]=="left_only"
        ]
        missing_events
```

```
Out[25]:
```

Season	EventName	RoundNumber	_merge
--------	-----------	-------------	--------

```
In [26]: laps_prepared=laps.copy()
        compound=laps_prepared["Compound"].astype("string").str.strip()

        laps_prepared["Compound"]=compound.mask(
        compound.str.lower().isin(["", "none", "nan", "null", "<na>"]),
        pd.NA
        )

        laps_prepared["LapTimeSeconds"]=(
        laps_prepared["LapTime"].dt.total_seconds()
        )

        laps_prepared["MissingLapTime"] = laps_prepared["LapTime"].isna()
        laps_prepared["MissingTyreLife"] = laps_prepared["TyreLife"].isna()
        laps_prepared["MissingStint"] = laps_prepared["Stint"].isna()
        laps_prepared["MissingPosition"] = laps_prepared["Position"].isna()
        laps_prepared["MissingCompound"] = laps_prepared["Compound"].isna()

        laps_prepared["IsPitLap"] = (
        laps_prepared["PitInTime"].notna()
        | laps_prepared["PitOutTime"].notna() #or
        )

        lap_keys=["Season", "RoundNumber", "DriverNumber", "LapNumber"]
```

```
laps_prepared["DuplicateLapKey"] = laps_prepared.duplicated(
    subset=lap_keys,
    keep=False
)

quality_attributes=[
    "MissingLapTime",
    "MissingTyreLife",
    "MissingStint",
    "MissingPosition",
    "MissingCompound",
    "IsPitLap",
    "DuplicateLapKey"
]

quality_summary=(
    laps_prepared[quality_attributes].sum().rename_axis("Finding").reset_index(name="Count")
)

display(quality_summary)

assert len(laps_prepared) == len(laps)
assert not laps_prepared["DuplicateLapKey"].any()
```

	Finding	Count
0	MissingLapTime	1539
1	MissingTyreLife	989
2	MissingStint	354
3	MissingPosition	135
4	MissingCompound	780
5	IsPitLap	6741
6	DuplicateLapKey	0

How stable are consecutive lap times?

The project aims to predict a driver's next lap time. Before modelling, I need to understand how lap times change between consecutive laps. From this, I will understand if the model is useful in real life, as it may have a great performance, but be useless in real life scenarios.

```
In [27]: training_pairs = laps_prepared[
    laps_prepared["Season"].isin([2022, 2023])
].copy()

keys = ["Season", "RoundNumber", "DriverNumber"]
training_pairs = training_pairs.sort_values(keys + ["LapNumber"])
groups = training_pairs.groupby(keys)
```

```

training_pairs["NextLapNumber"] = groups["LapNumber"].shift(-1)
training_pairs["NextLapTimeSeconds"] = groups["LapTimeSeconds"].shift(-1)

training_pairs["NextLapIsPitLap"] = groups["IsPitLap"].shift(-1)
training_pairs["NextLapTrackStatus"] = groups["TrackStatus"].shift(-1)
training_pairs["NextLapIsAccurate"] = groups["IsAccurate"].shift(-1)

training_pairs = training_pairs.loc[
    training_pairs["NextLapNumber"].eq(training_pairs["LapNumber"] + 1)
].dropna(subset=["LapTimeSeconds", "NextLapTimeSeconds"]).copy()

training_pairs["LapTimeChange"] = (
    training_pairs["NextLapTimeSeconds"] - training_pairs["LapTimeSeconds"]
)

training_pairs["AbsoluteLapTimeChange"] = (
    training_pairs["LapTimeChange"].abs()
)

training_pairs[["LapTimeChange", "AbsoluteLapTimeChange"]].describe(
    percentiles=[0.5, 0.9, 0.95, 0.99]
)

```

Out[27]:

	LapTimeChange	AbsoluteLapTimeChange
count	45710.000000	45710.000000
mean	-0.193228	2.898010
std	7.763536	7.204941
min	-96.859000	0.000000
50%	-0.006000	0.325000
90%	1.596000	10.892300
95%	11.250850	19.568100
99%	25.104380	35.894280
max	61.267000	96.859000

In [28]:

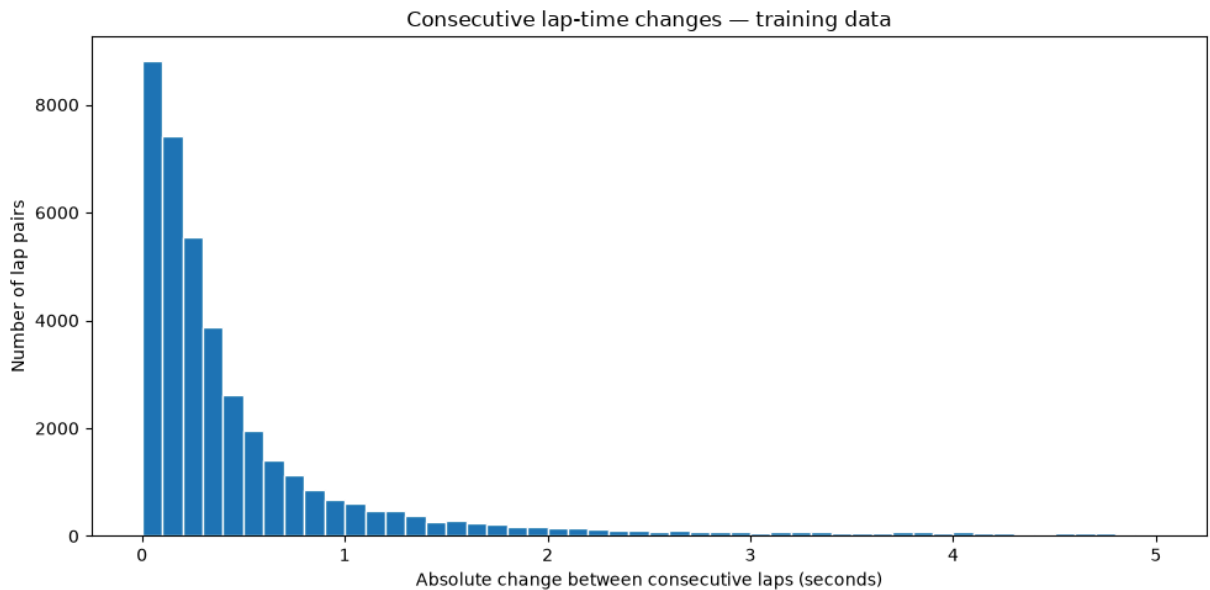
```

changes = training_pairs["AbsoluteLapTimeChange"]

plt.figure(figsize=(10, 5))
plt.hist(changes, bins=50, range=(0, 5), edgecolor="white")
plt.xlabel("Absolute change between consecutive laps (seconds)")
plt.ylabel("Number of lap pairs")
plt.title("Consecutive lap-time changes – training data")
plt.tight_layout()
plt.show()

print(f"Pairs outside the displayed range (>5 seconds): {changes.gt(5).mean():.1%}")

```



Pairs outside the displayed range (>5 seconds): 12.9%

Half of the absolute lap changes are no greater than 0.325 seconds, however, the mean absolute change is quite high, 2.898 seconds and the largest one being 96.859, being a weird value, maybe due to a safety car. This indicates that there are some large values that affect this mean. These pairs include race interruptions and pit laps, so these mean values are quite altered.

Selecting uninterrupted race-pace pairs

Pit entries and exits, interruptions and timing-quality issues can produce changes that differ from normal race pace. For the training analysis below, I keep consecutive laps from the same driver and race with recorded times, no pit entry or exit on either lap, green track status (`TrackStatus == "1"`) on both laps, and both FastF1 `IsAccurate` flags set to True. `IsAccurate` is a timing-quality flag supplied by FastF1, not a result of my own filters.

The consecutive-lap and recorded-time checks have already been applied to `training_pairs`. The following cell applies the remaining checks and creates `race_pace_pairs` for the later seasonality analysis. The race-example plots below exclude only pit laps, so they can still show interruptions.

These rules define the scope of the initial forecast evaluation: uninterrupted race pace. They do not mean that interruptions are unimportant. Checks on the next lap select historical pairs for analysis and evaluation; they are not information available to the model when making a prediction.

```
In [29]: race_pace_pairs = training_pairs.loc[
    training_pairs["IsPitLap"].eq(False)
    & training_pairs["NextLapIsPitLap"].eq(False)
    & training_pairs["TrackStatus"].eq("1")
    & training_pairs["NextLapTrackStatus"].eq("1")
```

```

    & training_pairs["IsAccurate"].eq(True)
    & training_pairs["NextLapIsAccurate"].eq(True)
].copy()

print(f"Training pairs before race-pace filters: {len(training_pairs):,}")
print(f"Training pairs after race-pace filters: {len(race_pace_pairs):,}")

```

Training pairs before race-pace filters: 45,710

Training pairs after race-pace filters: 35,873

Race Examples

Loading Race Data

```

In [30]: session = fastf1.get_session(2024, "Monza", "R")
        session.load(telemetry=False)

        race_laps = session.laps.copy()
        race_laps["LapTimeSeconds"] = race_laps["LapTime"].dt.total_seconds()

```

```

core      INFO      Loading data for Italian Grand Prix - Race [v3.8.3]
req       INFO      Using cached data for session_info
req       INFO      Using cached data for driver_info
req       INFO      Using cached data for session_status_data
req       INFO      Using cached data for lap_count
req       INFO      Using cached data for track_status_data
req       INFO      Using cached data for _extended_timing_data
req       INFO      Using cached data for timing_app_data
core      INFO      Processing timing data...
req       INFO      Using cached data for weather_data
req       INFO      Using cached data for race_control_messages
core      INFO      Finished loading data for 20 drivers: ['16', '81', '4', '5',
5', '44', '1', '63', '11', '23', '20', '14', '43', '3', '31', '10', '77', '27', '2
4', '18', '22']

```

Lap time evolution for three drivers

```

In [31]: drivers=["LEC", "VER", "ALO"]

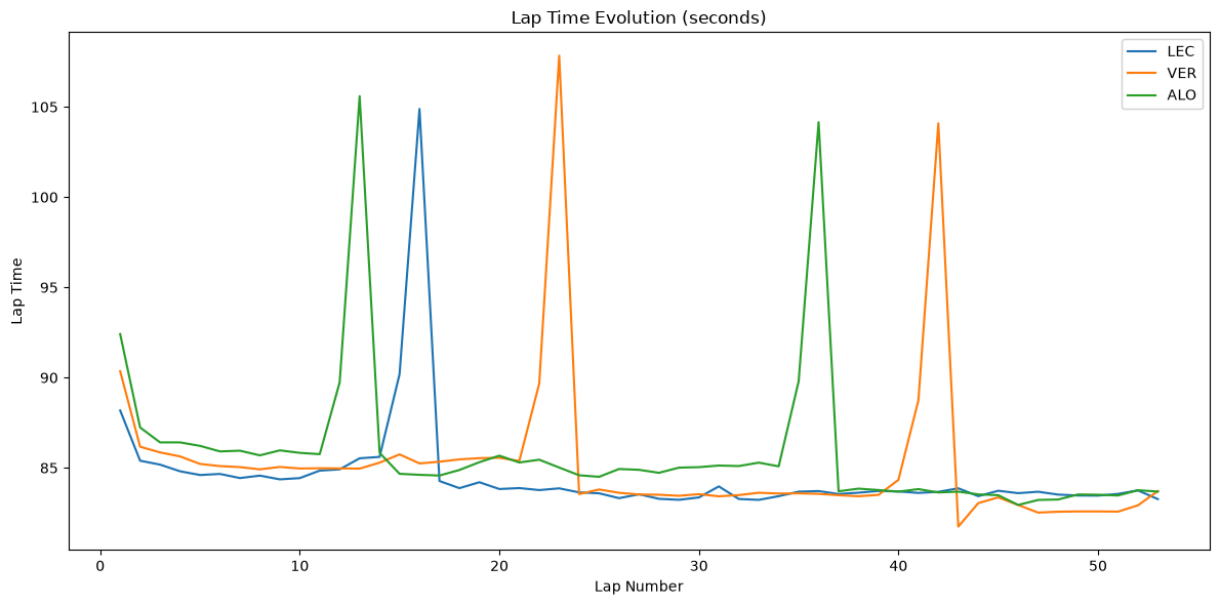
        plt.figure(figsize=(12,6))

        for driver in drivers:
            driver_laps=race_laps[race_laps["Driver"]==driver]
            plt.plot(
                driver_laps["LapNumber"],
                driver_laps["LapTimeSeconds"],
                label=driver
            )

        plt.xlabel("Lap Number")
        plt.ylabel("Lap Time")
        plt.title("Lap Time Evolution (seconds)")

```

```
plt.legend()
plt.tight_layout()
plt.show()
```



Observations: The plot includes pit laps, which can produce large lap-time peaks. The next plot excludes pit entries and exits to make the pace easier to see.

Lap time evolution excluding pit laps

```
In [32]: drivers = ["LEC", "VER", "ALO"]

fig, axes = plt.subplots(
    3, 1, figsize=(12, 10),
    sharex=True, sharey=True
)

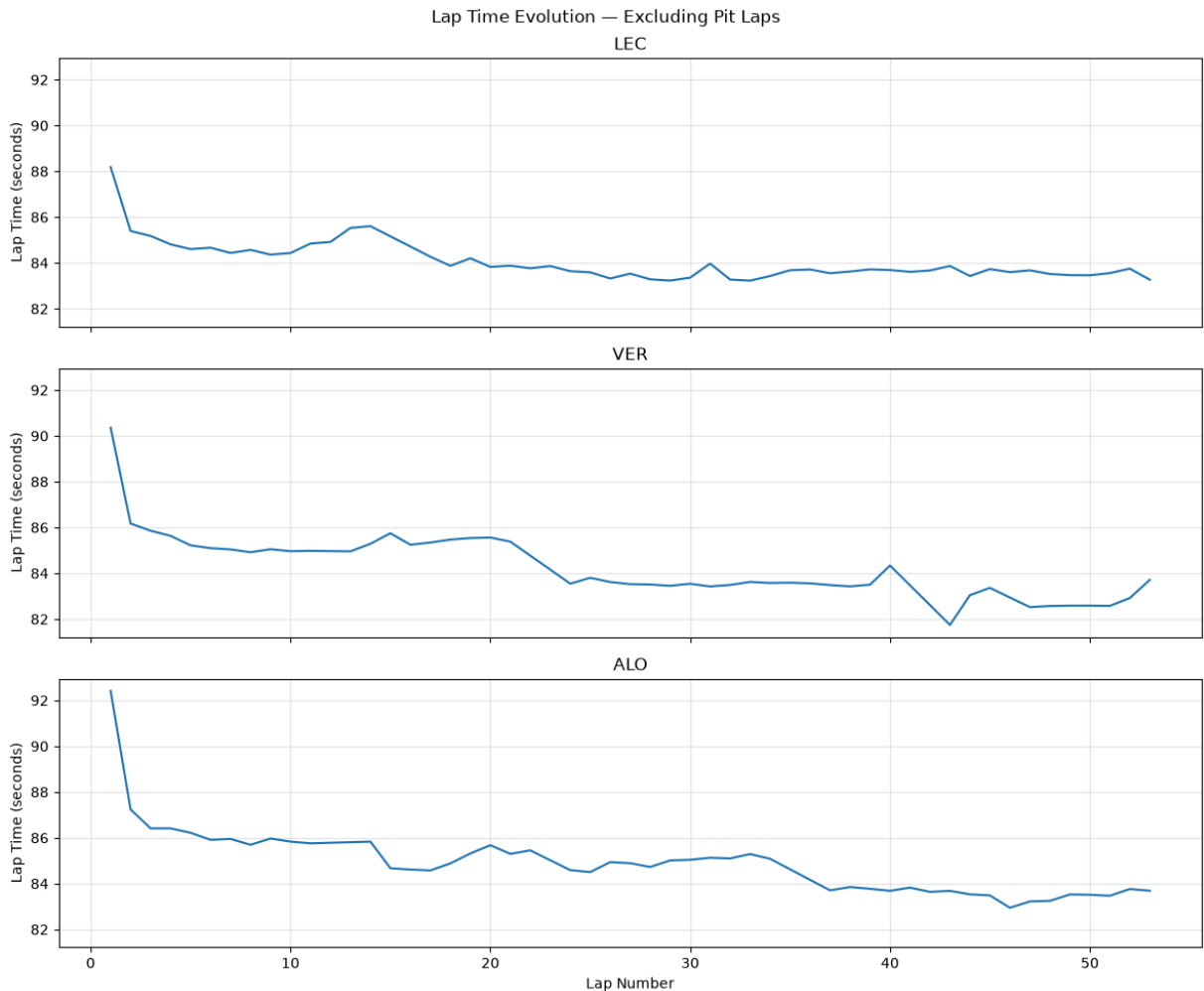
for driver, ax in zip(drivers, axes):
    driver_laps = race_laps[
        (race_laps["Driver"] == driver)
        & race_laps["PitInTime"].isna()
        & race_laps["PitOutTime"].isna()
    ].sort_values("LapNumber")

    ax.plot(
        driver_laps["LapNumber"],
        driver_laps["LapTimeSeconds"]
    )

    ax.set_title(driver)
    ax.set_ylabel("Lap Time (seconds)")
    ax.grid(alpha=0.3)

axes[-1].set_xlabel("Lap Number")
fig.suptitle("Lap Time Evolution – Excluding Pit Laps")
```

```
plt.tight_layout()
plt.show()
```



Observation: For these three drivers, lap times generally decrease early in the race and become more stable later. This visual pattern alone does not establish which model will predict the next lap best.

Tyre age and stints: one Monza example

For Leclerc at Monza 2024, I compare lap times against tyre age within each stint (the period between tyre changes). Each line identifies a stint and its compound. This gives context about how pace changes during a tyre run, beyond simply looking at lap number.

This is an illustrative example from the validation season. It is not used to select model settings; training-season analysis and validation performance guide those decisions.

```
In [33]: tyre_example = race_laps[
    race_laps["Driver"].eq("LEC")
].sort_values("LapNumber").copy()

no_pit_laps = (
    tyre_example["PitInTime"].isna()
```

```

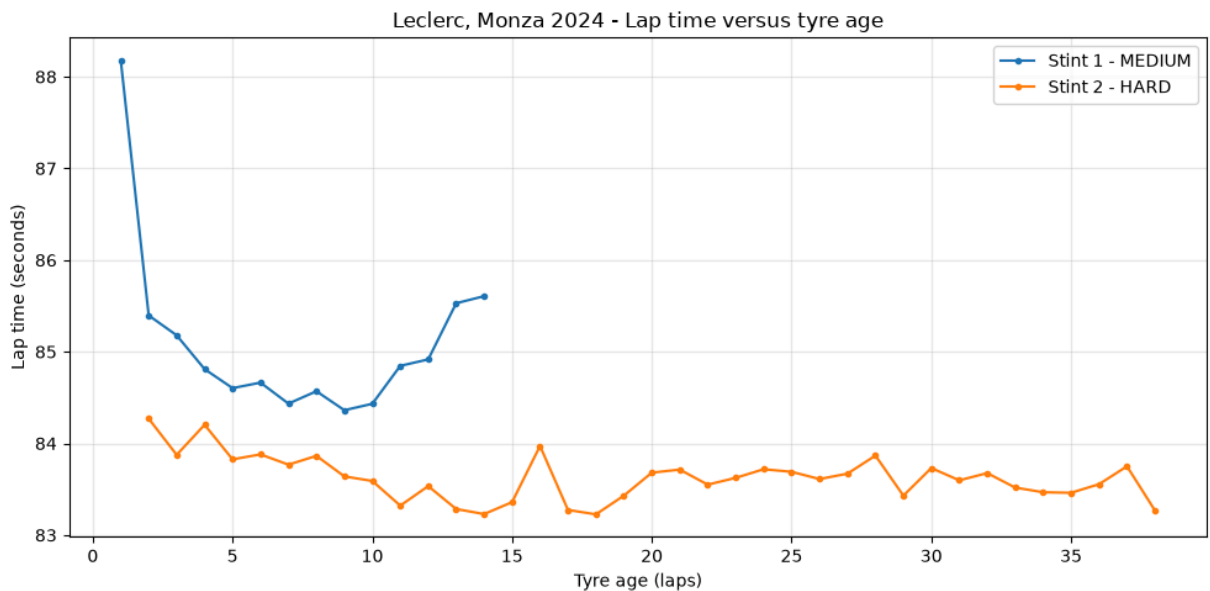
        & tyre_example["PitOutTime"].isna()
    )
    tyre_example["PlotLapTimeSeconds"] = tyre_example["LapTimeSeconds"].where(no_pit_la

plt.figure(figsize=(10, 5))

for stint, stint_laps in tyre_example.groupby("Stint"):
    compound = stint_laps["Compound"].iloc[0]
    plt.plot(
        stint_laps["TyreLife"],
        stint_laps["PlotLapTimeSeconds"],
        marker=".",
        label=f"Stint {int(stint)} - {compound}"
    )

plt.xlabel("Tyre age (laps)")
plt.ylabel("Lap time (seconds)")
plt.title("Leclerc, Monza 2024 - Lap time versus tyre age")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



Interpretation: We can see that in the first stint, towards the end on it, the lap times started increasing, since the tyres were in bad conditions. After the pit stop we can see a big improvement on lap times, however maintaining the time between 83 and 84 seconds

Monza observations

For the three selected drivers at Monza 2024, lap times generally decreased early in the race and became more stable later. Excluding pit laps made these patterns clearer. The single-driver tyre-age example adds context about pace within stints, but does not isolate tyre degradation from race progression or other factors.

These observations motivate comparisons with other races. They do not establish the causes of changes or the best forecasting model. The initial model will evaluate how much past lap times alone can predict, while tyre and race conditions remain possible sources of additional information.

Additional race examples

Bahrain 2022 and the Dutch Grand Prix 2023 extend the Monza example to two other circuits using training-season data. The same three drivers and pit-entry/exit exclusions are used for comparison. These plots show whether the pace patterns observed at Monza also appear in other races; they do not establish how every race behaves.

Only pit laps are excluded here, as in the Monza comparison. Other interruptions may still affect lap times. As in the Monza plots, pit laps are removed before plotting and the remaining points are connected.

Bahrain 2022 Lap time evolution excluding pit laps

```
In [39]: additional_race_laps = laps_prepared[
        laps_prepared["Season"].eq(2022)
        & laps_prepared["EventName"].eq("Bahrain Grand Prix")
    ].copy()

drivers = ["LEC", "VER", "ALO"]

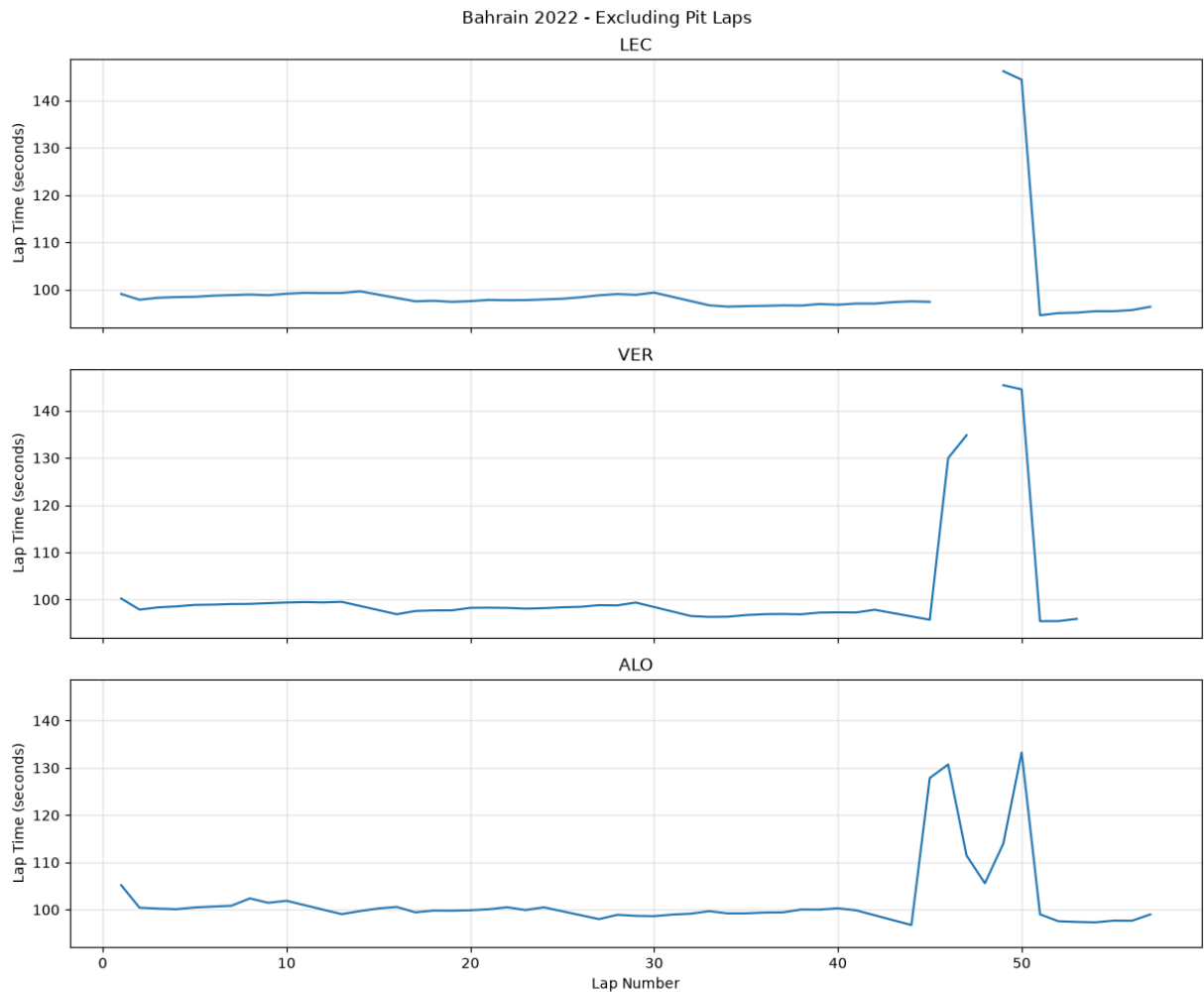
fig, axes = plt.subplots(
    3, 1, figsize=(12, 10),
    sharex=True, sharey=True
)

for driver, ax in zip(drivers, axes):
    example_driver_laps = additional_race_laps[
        (additional_race_laps["Driver"] == driver)
        & additional_race_laps["PitInTime"].isna()
        & additional_race_laps["PitOutTime"].isna()
    ].sort_values("LapNumber")

    ax.plot(
        example_driver_laps["LapNumber"],
        example_driver_laps["LapTimeSeconds"]
    )

    ax.set_title(driver)
    ax.set_ylabel("Lap Time (seconds)")
    ax.grid(alpha=0.3)

axes[-1].set_xlabel("Lap Number")
fig.suptitle("Bahrain 2022 - Excluding Pit Laps")
plt.tight_layout()
plt.show()
```



Dutch Grand Prix 2023 Lap time evolution excluding pit laps

```
In [40]: additional_race_laps = laps_prepared[
    laps_prepared["Season"].eq(2023)
    & laps_prepared["EventName"].eq("Dutch Grand Prix")
].copy()

drivers = ["LEC", "VER", "ALO"]

fig, axes = plt.subplots(
    3, 1, figsize=(12, 10),
    sharex=True, sharey=True
)

for driver, ax in zip(drivers, axes):
    example_driver_laps = additional_race_laps[
        (additional_race_laps["Driver"] == driver)
        & additional_race_laps["PitInTime"].isna()
        & additional_race_laps["PitOutTime"].isna()
    ].sort_values("LapNumber")

    ax.plot(
        example_driver_laps["LapNumber"],
        example_driver_laps["LapTimeSeconds"]
```

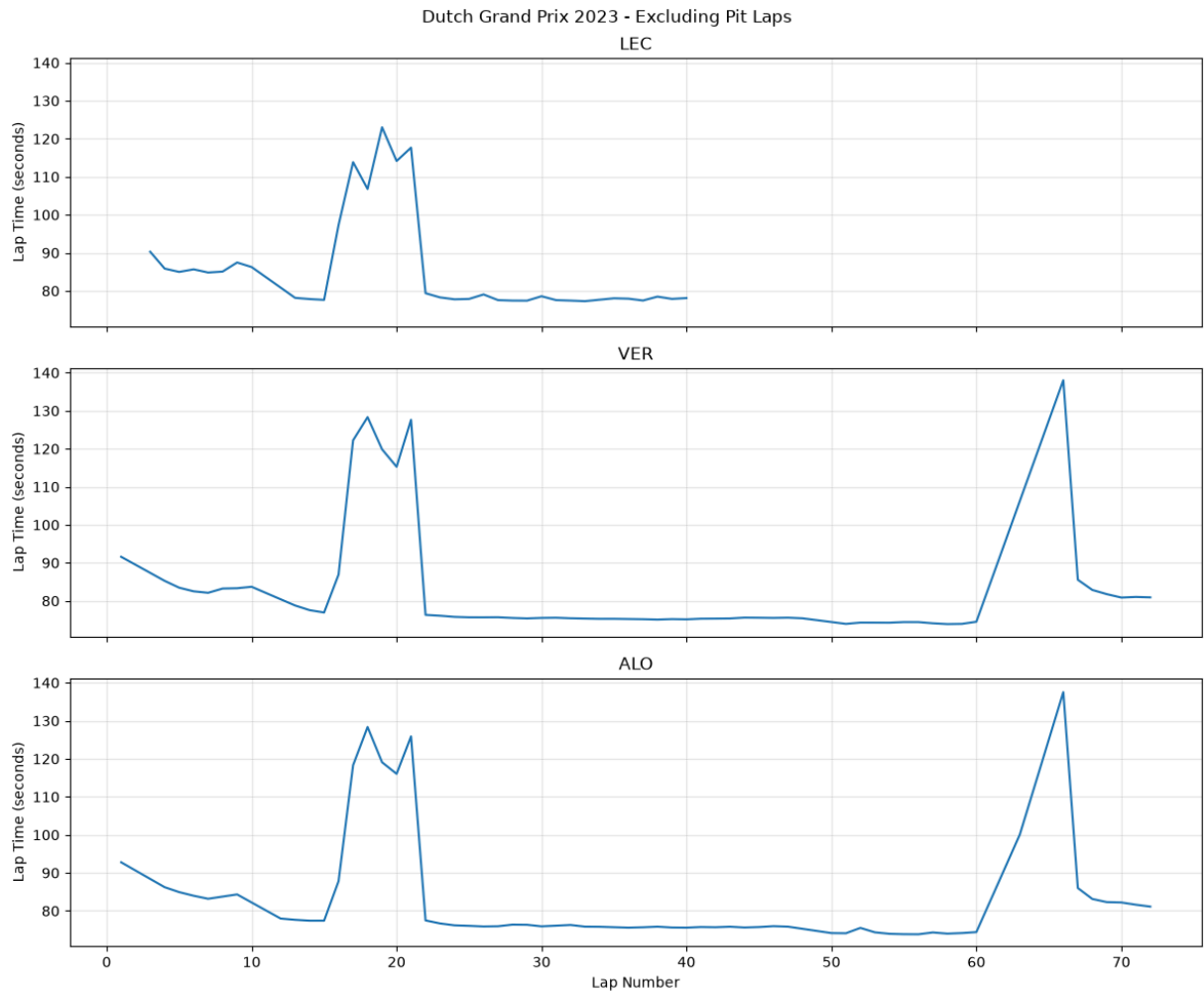
```

)

ax.set_title(driver)
ax.set_ylabel("Lap Time (seconds)")
ax.grid(alpha=0.3)

axes[-1].set_xlabel("Lap Number")
fig.suptitle("Dutch Grand Prix 2023 - Excluding Pit Laps")
plt.tight_layout()
plt.show()

```



What the EDA means for the first forecasting model

The EDA explores lap-time changes across drivers, races and tyre stints. Using only past lap times in the first model does not mean that tyre or race conditions are irrelevant.

Simple exponential smoothing provides a starting point: it estimates recent pace by giving more weight to recent lap times. It uses no explicit tyre, weather, trend or seasonal inputs. I will compare it with the naive forecast (the next lap equals the current lap) using MAE and RMSE on the same eligible pairs. A visible pattern alone does not establish which model forecasts best.

Data from 2022-2023 are used for training, 2024 for validation and 2025 for the final test. If a persistent trend remains difficult to forecast, a model with a trend component can be investigated. Tyre age, compound and race conditions remain possible inputs for later models; any benefit must be checked through forecasting performance.

The selected race examples illustrate possible differences, but they do not represent every race or prove what causes changes. Performance on the filtered pairs describes uninterrupted race pace, not all race situations. Unusual values in Bahrain and the Dutch Grand Prix can be explained by unusual events such as safety cars. Leclerc in the Dutch Grand Prix retired early, making it seem like the data stopped.

Time-series analysis before model-choice

First, I need to analyse 4 components for the time-series forecasting models. Trend, seasonality, cycle and variation will be analysed

Trend

```
In [36]: drivers = ["LEC", "VER", "ALO"]

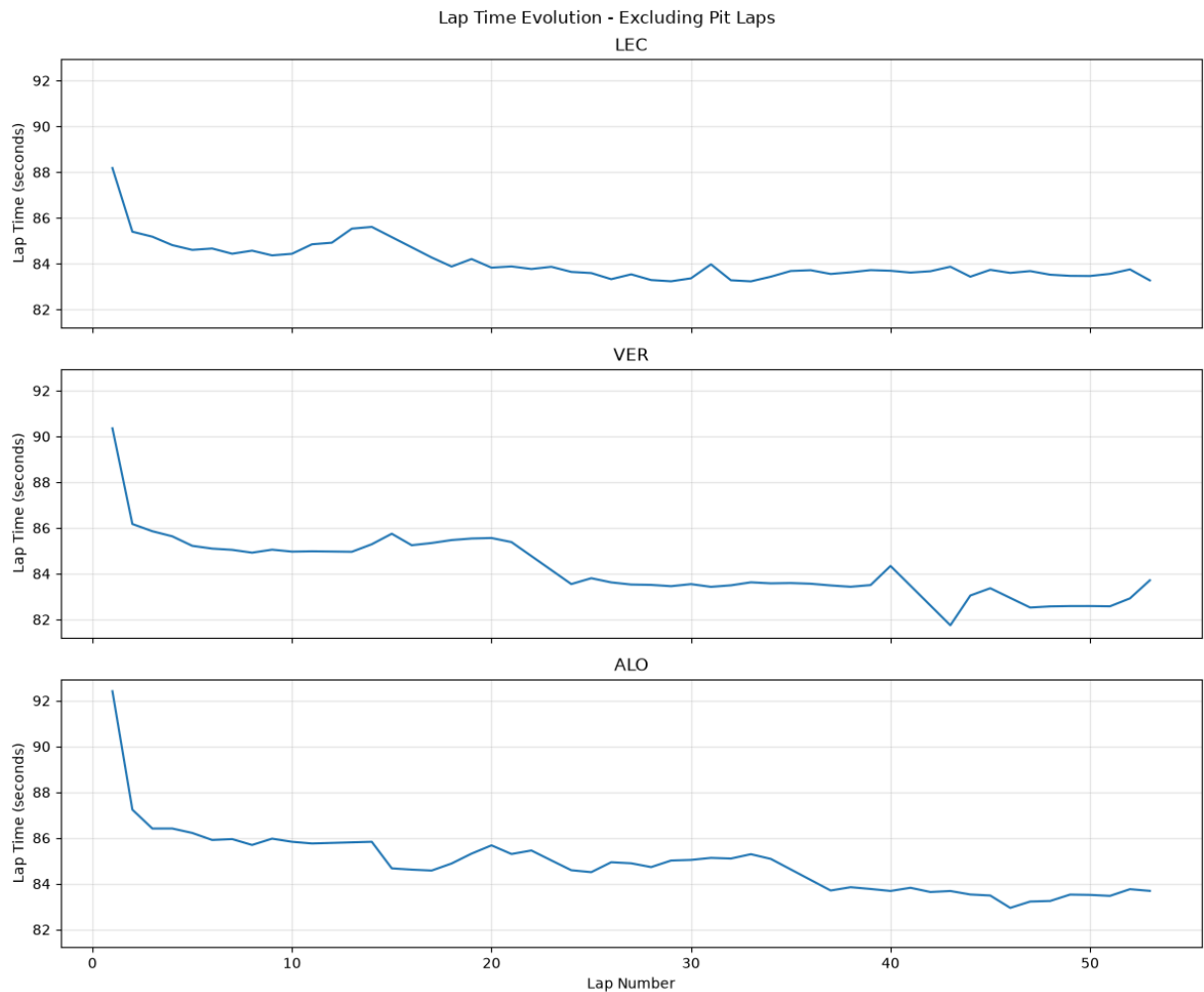
fig, axes = plt.subplots(
    3, 1, figsize=(12, 10),
    sharex=True, sharey=True
)

for driver, ax in zip(drivers, axes):
    driver_laps = race_laps[
        (race_laps["Driver"] == driver)
        & race_laps["PitInTime"].isna()
        & race_laps["PitOutTime"].isna()
    ].sort_values("LapNumber")

    ax.plot(
        driver_laps["LapNumber"],
        driver_laps["LapTimeSeconds"]
    )

    ax.set_title(driver)
    ax.set_ylabel("Lap Time (seconds)")
    ax.grid(alpha=0.3)

axes[-1].set_xlabel("Lap Number")
fig.suptitle("Lap Time Evolution - Excluding Pit Laps")
plt.tight_layout()
plt.show()
```



Seasonality

```
In [37]: from statsmodels.graphics.tsaplots import plot_acf

driver_laps = race_pace_pairs.loc[
    race_pace_pairs["Season"].eq(2023)
    & race_pace_pairs["EventName"].eq("Italian Grand Prix")
    & race_pace_pairs["Driver"].eq("LEC")
].sort_values("LapNumber")

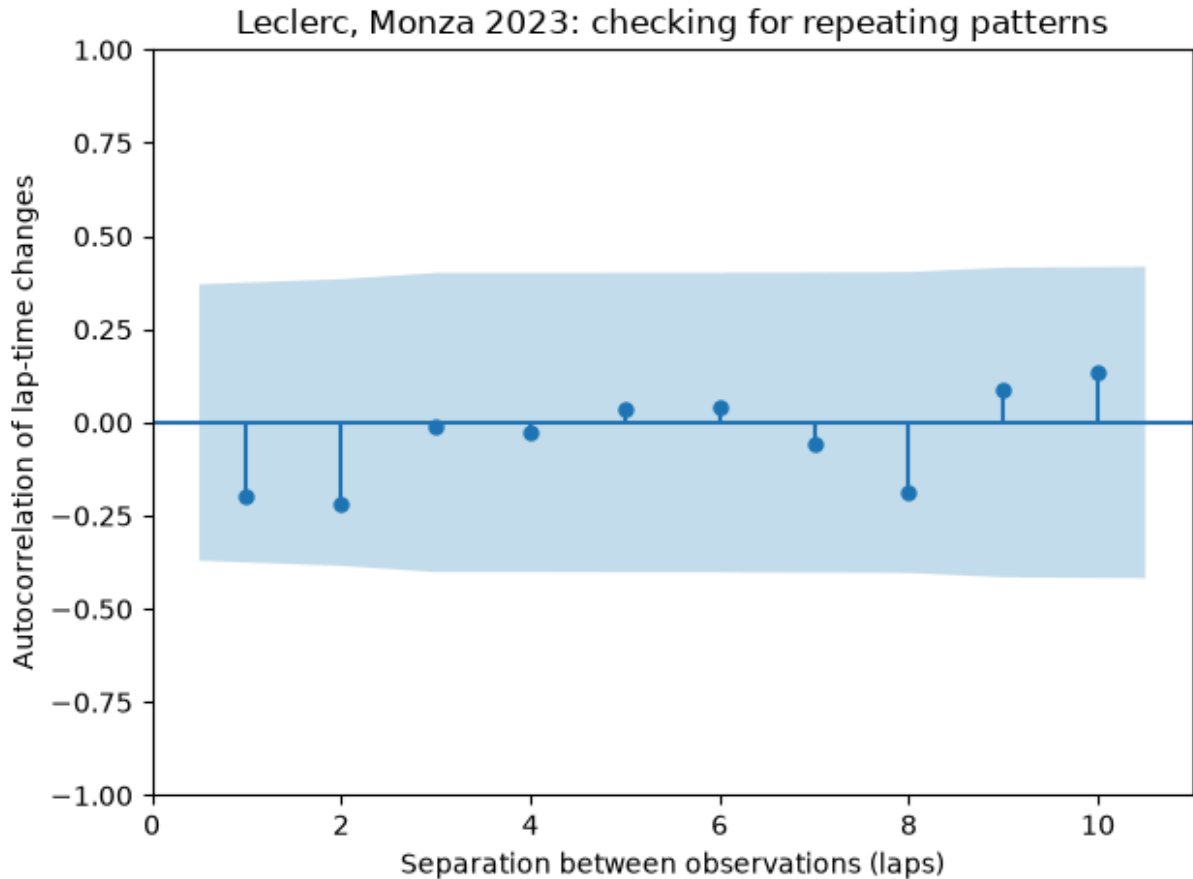
sequence = driver_laps["LapNumber"].diff().ne(1).cumsum()

longest = sequence.value_counts().idxmax()
lap_times = driver_laps.loc[
    sequence.eq(longest), "LapTimeSeconds"
]

lap_changes = lap_times.diff().dropna()

plot_acf(
    lap_changes,
    lags=min(10, len(lap_changes) // 2),
    zero=False
)
```

```
plt.xlabel("Separation between observations (laps)")
plt.ylabel("Autocorrelation of lap-time changes")
plt.title("Leclerc, Monza 2023: checking for repeating patterns")
plt.tight_layout()
plt.show()
```

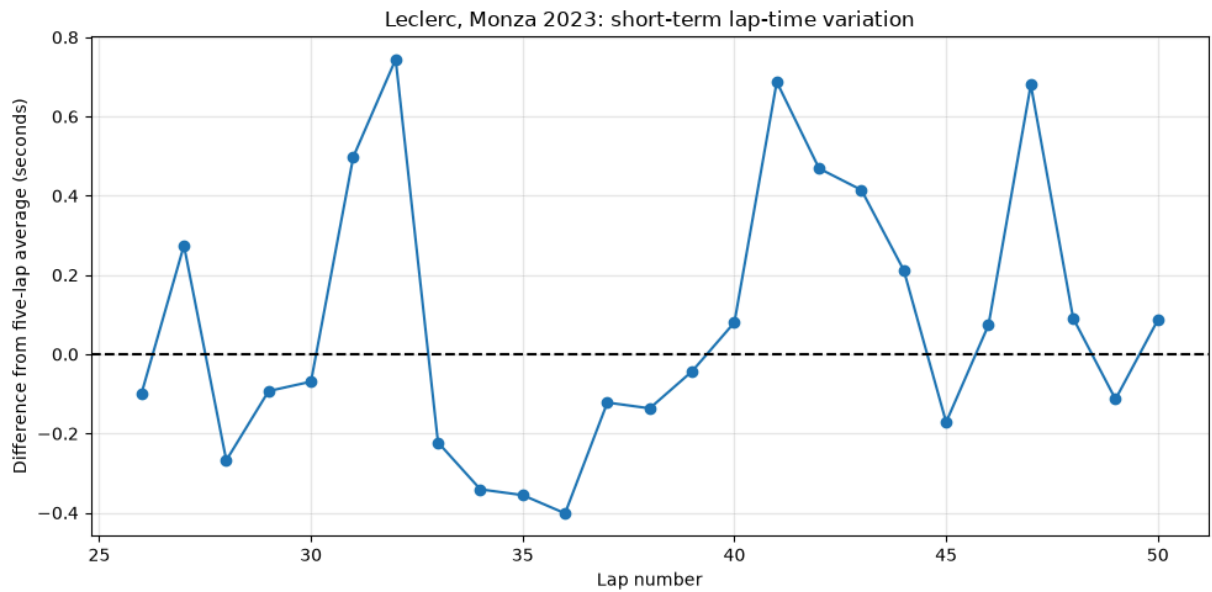


There is clearly not a seasonality in this data, as it is supposed to be as every lap in formula 1 lap is different

Irregular Variation

```
In [38]: local_average=lap_times.rolling(window=5).mean()
variation=lap_times - local_average
lap_numbers=driver_laps.loc[lap_times.index,"LapNumber"]

plt.figure(figsize=(10, 5))
plt.plot(lap_numbers, variation, marker="o")
plt.axhline(0, color="black", linestyle="--")
plt.xlabel("Lap number")
plt.ylabel("Difference from five-lap average (seconds)")
plt.title("Leclerc, Monza 2023: short-term lap-time variation")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



There are some variations in lap times comparing to the first 5 laps average. This is due to many variables, that cannot be proven here, but mainly driver's concentration, fuel load, tyre life, ammount of carss in track may affect these "weird" variations